

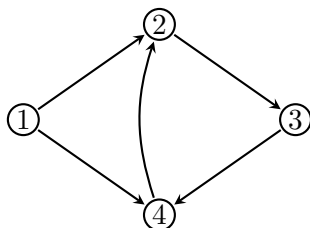
GRAFOVÉ ALGORITMY

1 Grafy a jejich reprezentace

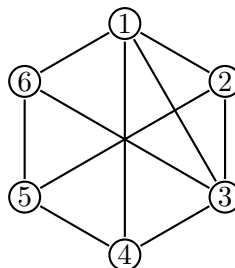
Grafy jsou základním stavebním kamenem mnoha úloh v informatice. Zachycují objekty a jejich vzájemné vztahy. Lze jimi reprezentovat třeba propojení zařízení v počítačové síti, schéma elektrického obvodu, mapu města nebo vazby mezi atomy. Jejich matematická podstata je však velice jednoduchá – *vrcholy* propojené *hranami*.

Definice 1.1 (Graf). Grafem G nazveme uspořádanou dvojici (V, E) , kde V je množina *vrcholů* (nebo také *uzlů*) a E množina *hran*, přičemž pokud

- $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$, pak G nazýváme *neorientovaným* grafem;
- $E \subseteq \{(u, v) \mid u, v \in V, u \neq v\}$, pak G nazýváme *orientovaným* grafem.



(a) Příklad orientovaného grafu.



(b) Příklad neorientovaného grafu.

1.1 Incidence, dosažitelnost a vzdálenost

Je-li $e \in E$ hrana a $v \in V$ jeden z jejích koncových vrcholů, říkáme, že e je s vrcholem v *incidentní*. V neorientovaném grafu jsou dva vrcholy sousední, vede-li mezi nimi hrana. V orientovaném grafu rozlišujeme směr: hrana (u, v) vychází z u a vstupuje do v .

Vrchol v je *dosažitelný* z vrcholu u , jestliže existuje cesta z u do v . Délkou cesty v neohodnoceném grafu rozumíme počet jejích hran. Vzdálenost

$$d(u, v)$$

je délka nejkratší cesty z u do v ; pokud taková cesta neexistuje, klademe $d(u, v) = \infty$. V neorientovaném grafu platí $d(u, v) = d(v, u)$, zatímco v orientovaném grafu se obě hodnoty mohou lišit a jedna z nich může být nekonečná.

Pojmy dosažitelnosti a vzdálenosti připravují dvě různé otázky. Prohledávání grafu může pouze zjistit, které vrcholy jsou ze startu dosažitelné, nebo může navíc hledat nejkratší cesty. BFS zvládne obě úlohy v neohodnoceném grafu; u ohodnocených hran budeme později potřebovat Dijkstrův algoritmus.

1.2 Stupeň vrcholu a princip sudosti

V neorientovaném grafu definujeme *stupeň vrcholu* v jako počet hran incidentních s v a značíme jej $\deg(v)$. U orientovaného grafu budeme v souladu s prezentacemi značit vstupní stupeň $\deg^+(v)$, výstupní stupeň

$\deg^-(v)$ a celkový stupeň

$$\deg(v) = \deg^+(v) + \deg^-(v).$$

Vstupní stupeň počítá hrany končící ve v , výstupní stupeň hrany, které z v vycházejí.

Tvrzení 1.2 (Princip sudosti). Pro každý konečný neorientovaný graf $G = (V, E)$ platí

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Proof. Při sčítání stupňů započítáme každou hranu právě dvakrát, jednou u každého jejího koncového vrcholu. Celkový součet je proto dvojnásobkem počtu hran. $\square$

Z principu sudosti plyne, že počet vrcholů lichého stupně je sudý. Součet všech stupňů je totiž sudý a součet lichého počtu lichých čísel by byl lichý. Tato jednoduchá podmínka bývá užitečná při kontrole, zda zadaná posloupnost stupňů vůbec může pocházet z nějakého grafu.

Poznámka 1.3. Za počátek teorie grafů se obvykle považuje Eulerovo řešení úlohy o sedmi mostech města Královec z roku 1736. Euler nahradil části města vrcholy a mosty hranami a ukázal, že existence požadované procházky závisí pouze na stupních vrcholů, nikoli na přesném tvaru mapy.

1.3 Reprezentace grafu

Podstatnou otázkou u grafů jsou způsoby jejich reprezentace, neboť při programování jsou různé způsoby různě efektivní a je dobré si mezi nimi umět vybrat. Pracujme tedy s grafem $G = (V, E)$, přičemž vrcholy si pro jednoduchost označíme přirozenými čísly $1, 2, \dots, n$, tzn. $V = \{1, 2, \dots, n\}$.

- **Matice sousednosti.** První způsob, jak graf reprezentovat, je pomocí matice. V tomto případě graf G popisuje matice $n \times n$, kde na pozici ij je 1, pokud z vrcholu i do vrcholu j vede hrana, jinak 0.

Příklad 1.4 (Prásátkový graf). Orientovaný graf G (viz obrázek 2) a jeho matice sousednosti $\mathbf{A}$.

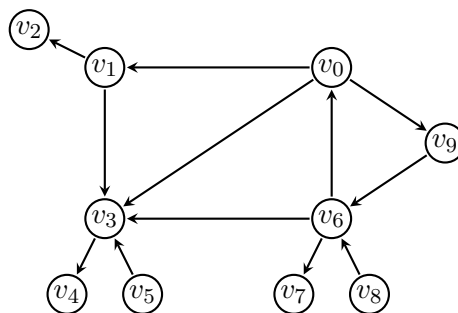


Figure 2: Prásátkový graf.

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- **Matice incidence.** Řádky jsou indexovány vrcholy a sloupce jednotlivými hranami. Na místě ij je 1, pokud hrana vede *do* vrcholu i , nebo -1 , pokud z něj vychází, jinak 0. Pro neorientované grafy píšeme vždy 1, pokud daná hrana vede *do/z* vrcholu.

Příklad 1.5 (Úplný graf K_4). Graf G má 4 vrcholy a celkem 6 hran (viz obrázek 3); jeho matici incidence označíme $\mathbf{I}$. Řádky představují (shora dolů) vrcholy 0, 1, 2, 3 a sloupce (zleva doprava) jednotlivé hrany $e_1, \dots, e_6$.

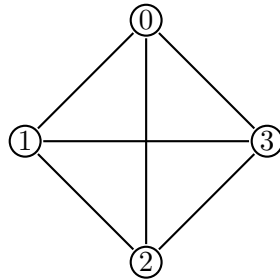
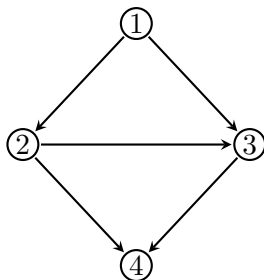


Figure 3: Úplný graf K_4 .

$$\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

- **Seznam sousedů.** Jedná se pravděpodobně o nejpoužívanější variantu reprezentace grafů. Pro každý vrchol $v \in V$ si jednoduše pamatujeme seznam $S[v]$ všech vrcholů, do kterých přímo vede hrana z vrcholu v .¹

Příklad 1.6. Graf G s vrcholy $1, \dots, 4$ a jejich seznamy sousedů S (viz obrázek 4).



$$\begin{aligned} S[1] &= \{2, 3\} & S[3] &= \{4\} \\ S[2] &= \{3, 4\} & S[4] &= \emptyset \end{aligned}$$

Figure 4: Graf G a seznamy sousedů pro jeho každý vrchol.

⁽¹⁾Název *seznam* zde odkazuje na použitou datovou strukturu. Z matematického pohledu popisuje $S[v]$ množinu sousedů.

Rozdíly shrnuje tabulka 1. Uvedené časy předpokládají přímou maticovou reprezentaci a obyčejný nesetříděný seznam sousedů. Jiná datová struktura může některou operaci urychlit za cenu větší paměti nebo složitější aktualizace.

Reprezentace	Paměť	Test hrany (u, v)	Výpis sousedů u
matice sousednosti	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
matice incidence	$\mathcal{O}(nm)$	$\mathcal{O}(m)$	$\mathcal{O}(nm)$
seznamy sousedů	$\mathcal{O}(n + m)$	$\mathcal{O}(\deg(u))$	$\mathcal{O}(\deg(u))$

Table 1: Základní porovnání reprezentací grafu.

Matice sousednosti je vhodná pro husté grafy a časté dotazy na existenci konkrétní hrany. Seznamy sousedů jsou přirozenou volbou pro řídké grafy a algoritmy, které postupně procházejí všechny hrany. Matice incidence se v běžné implementaci algoritmů používá méně, ale je důležitá v algebraickém popisu grafů a síťových úlohách. U neorientovaného grafu uloží seznamy sousedů každou hranu dvakrát, jednou u každého koncového vrcholu; konstantní násobek však v $\mathcal{O}$ -notaci nehraje roli.

2 Stromy

Stromy tvoří speciální oblast teorie grafů. V informatice jde o jeden z nejčastěji používaných typů grafů, a proto jim věnujeme samostatnou sekci, v níž si připomeneme některé základní pojmy.

Definice 2.1 (Strom). Neorientovaný graf $G = (V, E)$ je strom, pokud

- (i) je souvislý²
- (ii) a neobsahuje kružnici.

Je dobré si uvědomit, že mezi každou dvojicí vrcholů ve stromě vede **právě jedna** cesta. Zároveň z definice stromu plyne, že odebráním libovolné hrany se nutně poruší souvislost: přestane existovat cesta mezi koncovými vrcholy odebrané hrany.

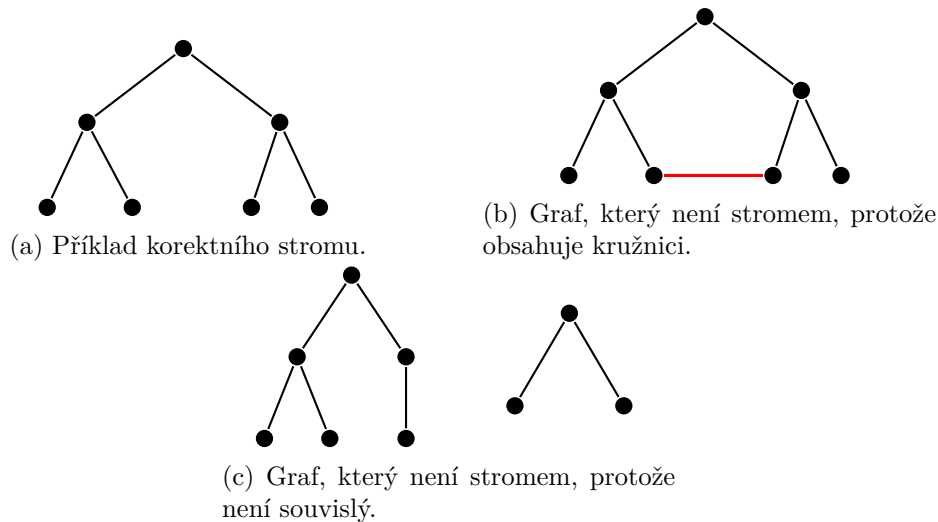
Tyto dvě vlastnosti nejsou jen důsledky definice, ale patří mezi několik rovnocenných způsobů, jak strom poznat. V praxi si proto můžeme vybrat tu charakterizaci, která se pro daný důkaz nebo algoritmus nejlépe hodí.

Věta 2.2 (Ekvivalentní charakterizace stromu). Pro neorientovaný graf $T = (V, E)$ jsou následující tvrzení ekvivalentní:

- (i) T je strom;
- (ii) mezi každými dvěma vrcholy vede právě jedna cesta;
- (iii) T je souvislý a odebráním libovolné hrany se stane nesouvislým;
- (iv) T neobsahuje kružnici a přidáním libovolné chybějící hrany vznikne právě jedna kružnice;
- (v) T je souvislý a platí $|E| = |V| - 1$.

Například poslední bod dovoluje rychlou kontrolu počtu hran, ale sám o sobě nestačí: graf s $|V| - 1$ hranami ještě nemusí být stromem, pokud není souvislý. Podobně graf bez kružnic může být lesem; teprve souvislost z něj udělá strom.

⁽²⁾Mezi každou dvojicí vrcholů vede cesta.



Poslední příklad 5c sice není z definice strom, avšak čtenář by mohl namítnout, že jednotlivé “části” daného grafu tvoří stromy. Tomuto typu grafů se říká (celkem pochopitelně) *lesy*, neboť jejich *komponenty souvislosti*³ tvoří stromy (viz obrázek 6).

Definice 2.3 (Les). Graf $G = (V, E)$ je les, pokud každá jeho komponenta souvislosti je strom.

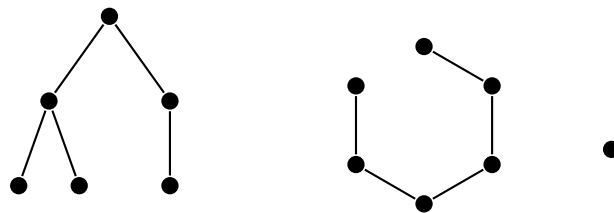


Figure 6: Příklad lesa.

Ekvivalentně můžeme les definovat jako *disjunktní sjednocení stromů*. Jako poslední si uvedme speciální typ stromu, se kterým budeme dále nejčastěji pracovat, a to tzv. *binární strom*. Blíže se s ním setkáme v sekci o haldě.

Definice 2.4 (Binární strom). Strom $T = (V, E)$ nazveme binární, pokud

- (i) je zakořeněný
- (ii) a každý vrchol má nejvýše dva syny.

Jeden z vrcholů stromu T je označen jako *kořen*. Typicky jej zakreslujeme nahoru nad všechny ostatní vrcholy a syny každého vrcholu umísťujeme vždy níže oba na stejnou úroveň (viz obrázek 7).

3 Prohledávání do šířky

Jednou ze základních úloh je procházení grafu z určitého vrcholu a zjištění dosažitelnosti ostatních vrcholů. Nejjednodušším algoritmem v tomto ohledu je tzv. *prohledávání do šířky* (anglicky *breadth-first search*, zkráceně BFS). Jeho základní princip spočívá v postupném objevování sousedů již nalezených

⁽³⁾Intuitivně se jedná o část grafu, kde mezi každými dvěma vrcholy vede cesta. Formální zavedení termínu se provádí přes *binární relaci*. Pro zájemce: [https://en.wikipedia.org/wiki/Component_\(graph_theory\)](https://en.wikipedia.org/wiki/Component_(graph_theory)).

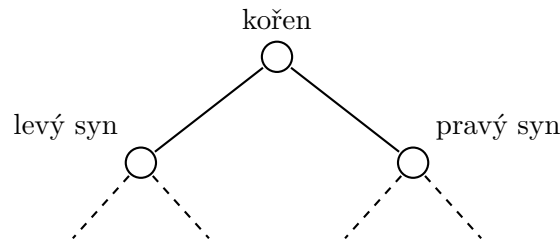


Figure 7: Příklad binárního stromu.

vrcholů. Na počátku dostaneme graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$. Postupně objevíme všechny sousedy vrcholu v_0 , poté všechny dosud nenalezené sousedy těchto vrcholů atd. Na BFS lze nahlížet tak, že do počátečního vrcholu nalijeme vodu a sledujeme, jak grafem postupuje vzniklá vlna.

Pro každý vrchol si budeme uchovávat jeho *stav*.

- *Nenalezený* – vrchol jsme ještě během výpočtu neviděli.
- *Otevřený* – vrchol jsme již našli, ale ještě jsme neprozkoumali všechny jeho sousedy.
- *Uzavřený* – vrchol jsme prozkoumali společně se všemi jeho sousedy a dál se jím již netřeba zabývat.

Na počátku začneme s jedním otevřeným vrcholem, a to v_0 (zde začínáme). Po prozkoumání všech jeho dosud nenalezených sousedů se jejich stav změní na otevřený a počáteční vrchol v_0 se uzavře. Obdobně pokračujeme pro nově otevřené vrcholy. Pokud by náhodou mezi dvojicí otevřených vrcholů existovala hrana, pak si sousedního vrcholu všimnat nebudeme, neboť byl již otevřen. Pro každý vrchol si ještě dodatečně můžeme uchovávat informaci, jak daleko se nachází od v_0 , co do počtu hran ležících na cestě.

Algoritmus 3.1: BFS (prohledávání do šířky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

// Inicializace

pro každý vrchol $v \in V$ **opakuj**

$stav(v) \leftarrow \text{nenalezený}$

$D(v) \leftarrow \infty$

$stav(v_0) \leftarrow \text{otevřený}$

$D(v_0) \leftarrow 0$

Založ frontu Q a přidej do ní vrchol v_0

dokud fronta Q je neprázdná **opakuj**

$v \leftarrow$ první vrchol ve frontě Q , který z ní odebereme

// Prozkoumáváme všechny dosud neobjevené sousedy

pro každý sousední vrchol w vrcholu v **opakuj**

pokud $stav(w) = \text{nenalezený}$ **pak**

$stav(w) \leftarrow \text{otevřený}$

$D(w) \leftarrow D(v) + 1$

Přidej w do fronty Q

$stav(v) \leftarrow \text{uzavřený}$

Výstup: Seznam vzdáleností D

3.1 Implementace v Pythonu

Program 3.1 reprezentuje graf slovníkem, jehož klíče jsou vrcholy a hodnoty jsou seznamy jejich sousedů. Frontu vytváří pomocí třídy `deque`: operace `popleft()` odebere nejdéle čekající vrchol, a proto jsou vrcholy zpracovány ve stejném pořadí, v jakém byly objeveny. Hodnota `None` v poli vzdáleností zde současně zastupuje stav *nenalezený*.

Vedle vzdálenosti si implementace ukládá také předchůdce každého objeveného vrcholu. Z těchto údajů lze zpětně sestavit nejkratší cestu: začneme v cílovém vrcholu a opakovaně přecházíme k jeho předchůdci, dokud nedojdeme do počátečního vrcholu. Vrcholy, které z počátku dosažitelné nejsou, si ponechají vzdálenost `None` a předchůdce `None`.

Program 3.1: Prohledávání grafu do šířky v Pythonu.

```
1  """Jednoduchá implementace prohledávání grafu do šířky."""
2
3  from collections import deque
4
5
6  def bfs(graph, start):
7      """Vrátí vzdálenosti a předchůdce vrcholu dosažitelných ze startu."""
8      distance = {vertex: None for vertex in graph}
9      parent = {vertex: None for vertex in graph}
10     queue = deque([start])
11     distance[start] = 0
12
13     while queue:
14         vertex = queue.popleft()
15         for neighbor in graph[vertex]:
16             if distance[neighbor] is None:
17                 distance[neighbor] = distance[vertex] + 1
18                 parent[neighbor] = vertex
19                 queue.append(neighbor)
20
21     return distance, parent
22
23
24  if __name__ == "__main__":
25     graph = {
26         "A": ["B", "C"],
27         "B": ["A", "D", "E"],
28         "C": ["A", "F"],
29         "D": ["B"],
30         "E": ["B", "F"],
31         "F": ["C", "E"],
32     }
33     distances, parents = bfs(graph, "A")
34     print("Vzdálenosti:", distances)
35     print("Předchůdci:", parents)
```

BFS rozděluje vrcholy do vrstev podle toho, v jaké vzdálenosti od počátečního vrcholu se nachází (viz obrázek 8). Nejdříve jsou prozkoumány vrcholy ve vzdálenosti 0 (tj. pouze v_0), poté ve vzdálenostech 1, 2, ... Z toho vyplývá, že kdykoliv při otevírání libovolného vrcholu v nastavujeme hodnotu $D(v)$, bude tato hodnota vždy odpovídat délce nejkratší cesty z v_0 do v . Tato hodnota bude však nastavena pouze u těch vrcholů, které jsou z v_0 dosažitelné (ostatní vrcholy zůstanou ve stavu *nenalezený*).

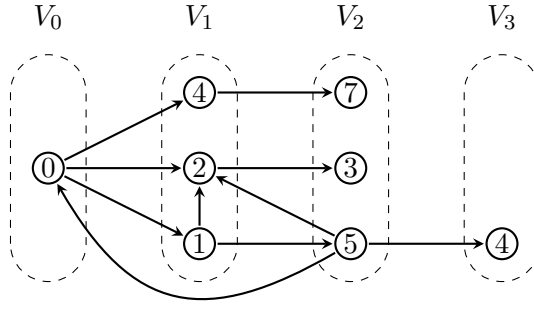


Figure 8: Vrcholy rozdělené do vrstev podle průběhu BFS.

Zbývá prozkoumat časovou a paměťovou složitost BFS. Označme si počet vrcholů grafu G na vstupu n a počet jeho hran m .

Věta 3.1 (Složitost BFS). Algoritmus BFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Proof. Inicializace potrvá $\mathcal{O}(n)$, neboť cyklus iteruje přes všechny vrcholy. Vnější cyklus provede maximálně n iterací, protože každý z vrcholů uzavřeme nejvýše jednou, tj. $\mathcal{O}(n)$.

S vnitřním cyklem je to trochu složitější, protože jeho počet iterací závisí na tom, který z vrcholů otevíráme (resp. na počtu jeho sousedů). Označme vrcholy $v_1, \dots, v_n$. Celkový počet iterací vnitřního cyklu přes všechny vrcholy bude $\sum_{i=1}^n \deg(v_i)$. Lze si ovšem všimnout jedné užitečné věci. Pokaždé, když prozkoumáváme sousední vrchol w nějakého vrcholu v , mohou nastat dva případy podle toho, jestli je G orientovaný graf, nebo neorientovaný.

- (i) Graf G je neorientovaný. Pak hranu, která spojuje v a w prozkoumáme právě *dvakrát* (jednou z vrcholu v a podruhé z vrcholu w). Každá hrana se tak započítá dvakrát, tzn. vnitřní cyklus se celkově provede max $2m$ -krát (po všech iteracích vnějšího cyklu).
- (ii) Graf G je orientovaný. Pak se hrana započítá pouze jednou, a to z vrcholu, z něhož vede. Celkově se vnitřní cyklus provede m -krát.

V prvním případě tak pro časovou složitost bude platit

$$\mathcal{O}\left(n + \sum_{i=1}^n \deg(v_i)\right) = \mathcal{O}(n + 2m) \stackrel{*}{=} \mathcal{O}(n + m).$$

(V kroku označeném $*$ zanedbáváme konstantu, neboť pracujeme s $\mathcal{O}$ -notací.)

V druhém případě dojdeme ke stejnému výsledku, akorát zmíněná suma bude, kvůli orientaci hran, rovna přesně počtu hran, tj.

$$\mathcal{O}\left(n + \sum_{i=1}^n \deg(v_i)\right) = \mathcal{O}(n + m).$$

Nyní k prostorové složitosti algoritmu. Na reprezentaci grafu (např. pomocí seznamu sousedů) spotřebujeme paměť $\mathcal{O}(n + m)$ (n vrcholů, m hran)⁴. Dále si uchováváme vrcholy ve frontě, v níž může být v jednu chvíli maximálně n vrcholů, tj. $\mathcal{O}(n)$, a k tomu máme seznamy $stav$ a D , oba s n položkami. Celkově spotřebujeme paměti

$$\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m).$$

⁽⁴⁾ Resp. každá hrana je v neorientovaných grafech započítána dvakrát, protože každý vrchol obsahuje v seznamu svých sousedů vždy *protější* vrchol (stejný argument jako u odvozování časové složitosti BFS).

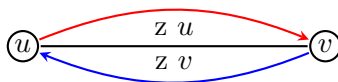


Figure 9: Znázornění situace v důkazu části (i) věty 3.1.

4 Prohledávání do hloubky

Na podobném přístupu jako BFS je založeno tzv. *prohledávání do hloubky* (anglicky *depth-first search*, zkráceně DFS). Vrcholy však tentokrát budeme zpracovávat rekurzivně. Pokaždé, když otevřeme nový vrchol, rekurzivně zpracujeme každého jeho dosud nenalezeného souseda. Po prozkoumání všech sousedů daný vrchol uzavřeme. Stejně jako u BFS si budeme uchovávat pole *stavů* jednotlivých vrcholů.

Algoritmus 4.1: DFS (prohledávání do hloubky)

Vstup: Graf $G = (V, E)$ a počáteční vrchol $v_0 \in V$

// Inicializace

pro každý vrchol $v \in V$ **opakuj**

$stav(v) \leftarrow \text{nenalezený}$

Zavolej DFS2(v_0)

// Rekurzivní volání na sousední vrcholy

Funkce DFS2(vrchol $v \in V$)

$stav(v) \leftarrow \text{otevřený}$

pro každý sousední vrchol w vrcholu v **opakuj**

pokud $stav(w) = \text{nenalezený}$ **pak**

Zavolej DFS2(w)

$stav(v) \leftarrow \text{uzavřený}$

4.1 Implementace v Pythonu

Program 4.1 zachovává rekurzivní podobu pseudokódu. Vnořená funkce `visit` odpovídá funkci DFS2: otevře zadaný vrchol, rekurzivně navštíví všechny jeho dosud nenalezené sousedy a nakonec jej uzavře. Seznam `order` proto zaznamenává pořadí prvního otevření vrcholů, nikoli pořadí jejich uzavření.

Implementace navíc ukládá časy otevření a uzavření a předchůdce v DFS stromu. Proměnná `time` je společná všem rekurzivním voláním, a proto je ve funkci `visit` označena klíčovým slovem `nonlocal`. Program zpracuje pouze komponentu obsahující počáteční vrchol. Chceme-li vytvořit DFS les celého nesouvislého grafu, po návratu z `visit(start)` zavoláme stejnou funkci také pro každý vrchol, který zůstal nenalezený.

Program 4.1: Rekurzivní prohledávání grafu do hloubky v Pythonu.

```

1  """Rekurzivní implementace prohledavani grafu do hloubky."""
2
3
4  def dfs(graph, start):
5      """Vrati poradí, casy otevreni a uzavreni a DFS strom."""
6      state = {vertex: "nenalezeny" for vertex in graph}
```

```

7   opened = {}
8   closed = {}
9   parent = {vertex: None for vertex in graph}
10  order = []
11  time = 0
12
13  def visit(vertex):
14      nonlocal time
15      state[vertex] = "otevreny"
16      time += 1
17      opened[vertex] = time
18      order.append(vertex)
19
20      for neighbor in graph[vertex]:
21          if state[neighbor] == "nenalezeny":
22              parent[neighbor] = vertex
23              visit(neighbor)
24
25      state[vertex] = "uzavreny"
26      time += 1
27      closed[vertex] = time
28
29  visit(start)
30  return order, opened, closed, parent
31
32
33  if __name__ == "__main__":
34      graph = {
35          "A": ["B", "C"],
36          "B": ["A", "D", "E"],
37          "C": ["A", "F"],
38          "D": ["B"],
39          "E": ["B", "F"],
40          "F": ["C", "E"],
41      }
42      result = dfs(graph, "A")
43      print("Poradi otevreni:", result[0])
44      print("Casy otevreni:", result[1])
45      print("Casy uzavreni:", result[2])
46      print("Predchudci:", result[3])

```

4.2 Časy otevření a uzavření vrcholů

Průběh DFS lze zachytit dvěma časovými údaji. Při prvním otevření vrcholu v si uložíme čas $in(v)$ a po zpracování všech jeho sousedů čas $out(v)$. Počítadlo zvýšíme při každé z těchto událostí, takže pro každý navštívený vrchol platí $in(v) < out(v)$.

Časové intervaly dvou vrcholů se buď nepřekrývají, nebo je jeden celý uvnitř druhého. Jestliže DFS otevře w během zpracování v , musí nejprve dokončit celé rekursivní volání pro w a teprve poté může uzavřít v . Dostáváme tedy

$$in(v) < in(w) < out(w) < out(v).$$

Tato vlastnost umožňuje z časů poznat vztah předka a potomka v DFS stromu, aniž bychom museli znovu procházet graf.

Hrany, po nichž DFS poprvé otevřelo nový vrchol, tvoří *DFS strom*; v nesouvislém grafu dostaneme DFS les. Strom popisuje strukturu rekurzivních volání a používá se například při hledání komponent souvislosti, mostů a artikulací. Most je hrana, jejímž odebráním se zvýší počet komponent grafu; časové údaje DFS dovolují rozhodnout, zda se podstrom umí vrátit zpět k některému předkovi jinou hranou.

Poznámka 4.1. Myšlenka systematického procházení do hloubky je starší než elektronické počítače a objevuje se už při řešení bludišť. Formální algoritmické použití DFS se rozšířilo zejména v 60. letech 20. století; Robert Tarjan na něm později založil několik lineárních algoritmů pro strukturu grafů.

Věta 4.2 (Složitost DFS). Algoritmus DFS doběhne v čase $\mathcal{O}(n + m)$ a spotřebuje paměť $\mathcal{O}(n + m)$.

Proof. Algoritmus DFS se oproti BFS liší pouze v pořadí, v jakém dosažitelné vrcholy zpracovává. To však nemá na časovou složitost žádný vliv. Argument pro její odvození je stejný jako u BFS, viz věta 3.1 v minulé sekci.

V paměti si musíme uchovávat reprezentaci grafu, to zabere $\mathcal{O}(n+m)$ paměti (opět např. pomocí seznamu sousedů) a máme seznam *stav* s n prvky (vrcholy). Zároveň si při volání DFS2 musíme na zásobník rekurze ukládat jednotlivé *aktivační záznamy*. Protože vrcholů je v grafu n , pak na zásobníku rekurze bude v jednu chvíli maximálně n záznamů, tj. $\mathcal{O}(n)$. Celkově spotřebujeme $\mathcal{O}(n + m) + \mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n + m)$ paměti. $\square$

Je dobré si uvědomit, že byť algoritmy BFS a DFS mají stejnou asymptotickou časovou i prostorovou složitost, nehodí se rovnocenně na stejné úlohy. BFS nalezne nejkratší cestu v neohodnoceném grafu. DFS se naopak často hodí tam, kde chceme nejprve prozkoumat jednu možnost opravdu do hloubky. Jeho zásobník obsahuje nejvýše jednu právě rozpracovanou cestu, zatímco fronta BFS může současně obsahovat celou širokou vrstvu grafu; v konkrétní úloze proto může DFS spotřebovat méně paměti, nejde však o obecnou asymptotickou záruku.

Cesta, kterou se DFS dostane do libovolného vrcholu v , nemusí být nutně nejkratší (silně závisí na pořadí, v jakém procházíme sousedy jednotlivých vrcholů).

5 Binární halda

Uvedme na úvod motivační příklad. Mějme seznam čísel, z něhož chceme co nejrychleji vybrat minimální nebo maximální hodnotu. Pro maximum bychom mohli sestavit následující jednoduchý algoritmus.

Algoritmus 5.1: Vyhledání maximální hodnoty v seznamu (MAX)

Vstup: Seznam čísel $x_1, x_2, \dots, x_n$

$m \leftarrow x_1$

pro $i = 2, 3, \dots, n$ **opakuji**

pokud $x_i > m$ **pak**

$m \leftarrow x_i$

vrať m

Výstup: Maximální hodnota seznamu m

Časová složitost bude zjevně $\mathcal{O}(n)$, neboť algoritmus prochází všech n prvků. Pokud ale se seznamem pracujeme opakovaně a průběžně jej udržujeme vhodně uspořádaný, můžeme minimum či maximum získat rychleji. K tomu můžeme použít tzv. *haldu*.

Definice 5.1 (Minimová binární halda). Minimová binární halda je datová struktura tvaru binárního stromu s množinou vrcholů V a klíčovou funkcí $\text{key} : V \rightarrow \mathbb{Z}$, kde je v každém vrcholu uložena *právě jedna* hodnota $\text{key}(v)$ (tzv. *klíč*) a navíc platí:

- (i) každá hladina je *plně obsazena*, kromě poslední, přičemž vrcholy poslední hladiny jsou zaplněny *zleva*,
- (ii) a je-li v libovolný vrchol a s jeho syn, pak $\text{key}(v) \leq \text{key}(s)$.

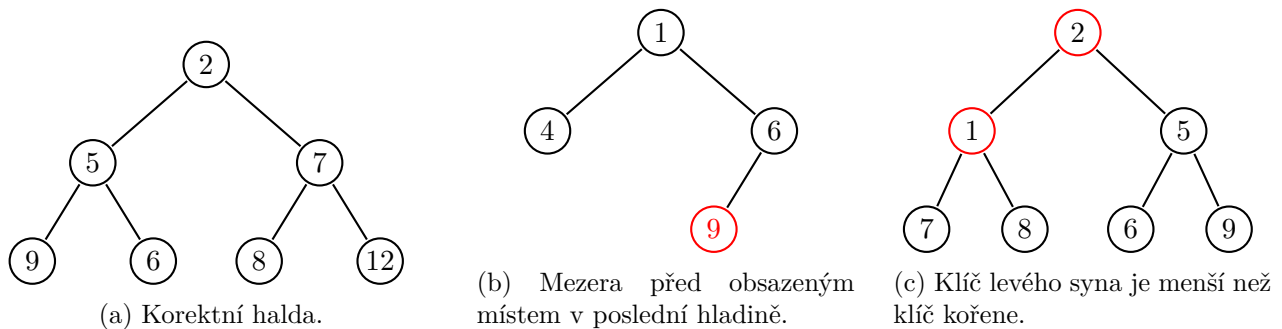


Figure 10: Příklady korektních a nekorektních hald.

Zde se na chvíli pozastavme. Podmínka (ii) v definici binární haldy 5.1 má velmi příjemný důsledek. Pokud se budeme pohybovat od kořene směrem dolů, hodnoty klíčů se nemohou zmenšovat. Minimum se proto vždy nachází *v kořeni stromu*, takže jeho zjištění zvládneme v *konstantním čase* $\mathcal{O}(1)$.

Pokud bychom chtěli naopak *maximovou binární haldu*, bude definice 5.1 vypadat obdobně, akorát v podmínce (ii) bude obrácená nerovnost, tj. $\text{key}(v) \geq \text{key}(s)$ (klíč v rodiči má vždy stejnou nebo vyšší hodnotu než klíče v jeho synech). Maximum se bude opět nacházet v kořeni haldy.

Je však více operací, které bychom rádi s haldou prováděli. Vypišme si všechny, které nás budou zajímat.

- $\text{INSERT}(x)$ – *vložení* nového vrcholu s klíčem x do haldy.
- $\text{MIN}()$ – *nalezení* vrcholu s nejmenším klíčem (už jsme zmínili).
- $\text{EXTRACTMIN}()$ – *odebrání* vrcholu s nejmenším klíčem.
- $\text{INCREASE}(v)$ – *zvýšení* hodnoty klíče ve zvoleném vrcholu v .
- $\text{DECREASE}(v)$ – *snížení* hodnoty klíče ve zvoleném vrcholu v .

5.1 Halda uložená v poli

Úplný tvar binární haldy dovoluje vynechat ukazatele mezi vrcholy a uložit klíče přímo do pole. Vrcholy očíslováme po hladinách zleva doprava, počínaje kořenem s indexem 1. Je-li vrchol na pozici i , pak jeho levý syn má index $2i$, pravý syn $2i + 1$ a rodič (pro $i > 1$) index $\lfloor i/2 \rfloor$.

Protože jsou všechny hladiny kromě poslední plné a poslední se zaplňuje zleva, nevznikají v poli žádná prázdná místa. První volná pozice je vždy konec pole, takže vložení nového klíče i odebrání posledního

vrcholu jsou jednoduché. Samotné obnovení haldového uspořádání pak probíhá prohazováním prvků na uvedených indexech.

V programovacích jazycích s indexací pole od nuly se vzorce posunou: děti prvku i leží na pozicích $2i + 1$ a $2i + 2$, rodič na $\lfloor (i - 1)/2 \rfloor$. Jde o stejnou datovou strukturu; před implementací je pouze nutné zvolit jednu konvenci a důsledně ji dodržet.

Podívejme se postupně na jednotlivé operace a jejich realizaci. Slovní popis vždy doplníme pseudokódem. V něm budeme haldu zapisovat jako pole $H[1], \dots, H[n]$ indexované od jedničky; proměnná n označuje jeho aktuální délku.

5.2 Vkládání nového vrcholu

Při vkládání nového vrcholu (INSERT) nejdříve potřebujeme rozhodnout, kam jej umístíme. S tím nám pomůže podmínka (i): nový vrchol vložíme na první volné místo v pořadí zleva doprava. Pokud je dosavadní poslední hladina plná, založíme novou hladinu a vrchol umístíme zcela vlevo. Tím zachováme tvar haldy, ale nemusí být splněna podmínka (ii). Klíč nového vrcholu může být menší než klíč jeho rodiče. Haldu opravíme postupným prohazováním klíče s rodičem, dokud nebude podmínka splněna (viz obrázek 11). Postup můžeme zapsat zkráceně ve třech bodech takto:

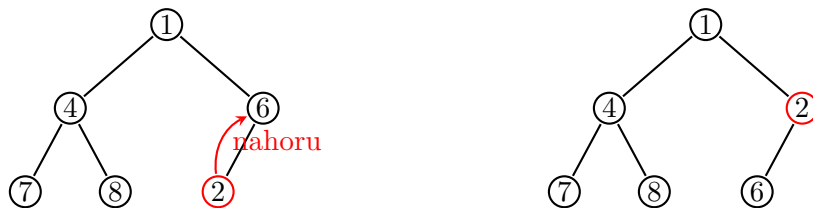


Figure 11: Vkládání nového vrcholu do haldy.

- Vložíme nový vrchol s klíčem x na první volnou pozici zleva na poslední hladině.
- pokud je klíč rodiče větší než x , prohodíme vrcholy (resp. jejich klíče)⁵.
- Opakujeme druhý krok.

Samotné probublávání nového klíče směrem ke kořeni oddělíme do pomocné operace BUBBLEUP. Díky tomu ji později použijeme beze změny také při snížení existujícího klíče.

Algoritmus 5.2: BUBBLE-UP

Vstup: Minimová halda $H[1], \dots, H[n]$ a index $i \in \{1, \dots, n\}$

dokud $i > 1$ a $H[\lfloor i/2 \rfloor] > H[i]$ **opakuj**

$r \leftarrow \lfloor i/2 \rfloor$ *

index rodiče Prohoď $H[i]$ a $H[r]$ $i \leftarrow r$

5.3 Odstranění minima

Získání minimálního klíče v haldě je triviální: přečteme klíč v kořeni a jsme hotovi. Pokud však chceme minimum operací EXTRACTMIN také odstranit, je to o něco složitější. Kořen nemůžeme jen smazat, jeho pozici musí zastoupit jiný vrchol. Abychom neporušili tvar haldy, přesuneme do kořene

⁽⁵⁾Vrchol v podstatě takto 'probublá' do správné pozice ve stromě (příp. až do pozice kořene).

poslední vrchol v pořadí zleva doprava, tedy nejpravější obsazený vrchol poslední hladiny, a jeho původní pozici odstraníme.

Nyní však (stejně jako u vkládání vrcholu) nastává problém s druhou podmínkou, neboť touto operací jsme ji mohli porušit. Provedeme obdobný proces, jako při vkládání vrcholu do haldy, ale vrchol nyní bude „probublávat“ směrem dolů. Podíváme se na syny daného vrcholu a pokud klíč některého z nich je menší než klíč nového kořene, pak jej s ním prohodíme (v případě, že klíče obou synů jsou menší, než klíč kořene, prohodíme s menším z nich). V bodech můžeme zapsat celou operaci následovně:

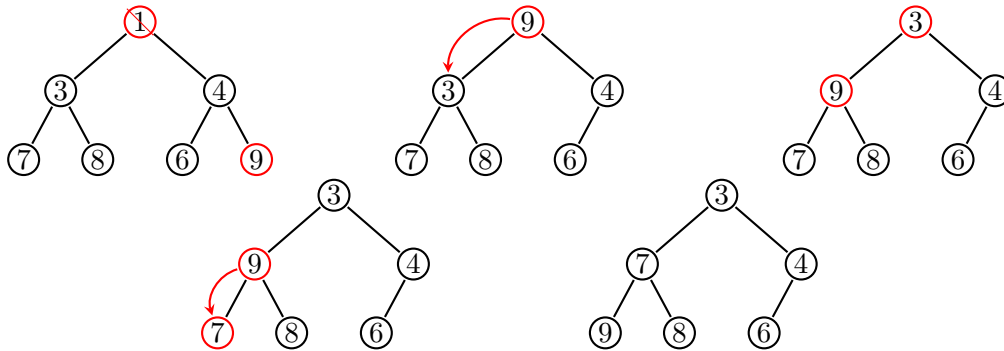


Figure 12: Odebírání vrcholu (kořene) s minimálním klíčem.

- Smažeme kořen a nahradíme jej nejpravějším obsazeným vrcholem na poslední hladině.
- Tento vrchol (resp. jeho klíč) porovnáme s jeho syny a případně prohodíme s tím, jehož klíč je menší.
- Opakujeme druhý krok.

Operace, při nichž vrchol během vkládání „probublává“ nahoru nebo během odebírání minima dolů, se někdy nazývají BUBBLEUP a BUBBLEDOWN. Ještě se nám budou hodit u operací INCREASE a DECREASE.

Při probublávání dolů musíme v každém kroku vybrat menšího ze synů. Prohození s větším synem by nemuselo obnovit haldovou podmínku vůči tomu menšímu. Pokud pravý syn neexistuje, porovnáváme pouze s levým.

Algoritmus 5.3: BUBBLE-DOWN

Vstup: Minimová halda $H[1], \dots, H[n]$ a index $i \in \{1, \dots, n\}$

dokud $2i \leq n$ **opakuji**

$s \leftarrow 2i // *$

zatím levý syn **pokud** $2i + 1 \leq n$ a $H[2i + 1] < H[s]$ **pak**

$s \leftarrow 2i + 1 // *$

pravý syn je menší

pokud $H[i] \leq H[s]$ **pak**

vrať

Prohoď $H[i]$ a $H[s]$ $i \leftarrow s$

Pomocí této operace už lze EXTRACTMIN zapsat velmi stručně. Zvlášť ošetřujeme jednoprvkovou haldu: po odebrání posledního prvku již není co probublávat.

Algoritmus 5.4: EXTRACT-MIN**Vstup:** Neprázdná minimová halda $H[1], \dots, H[n]$ $m \leftarrow H[1]$ $H[1] \leftarrow H[n]$ Odstraň poslední prvek pole H $n \leftarrow n - 1$ **pokud** $n > 0$ **pak** Zavolej BUBBLE-DOWN($H, 1$)**vrať** m **Výstup:** Odebraný minimální klíč m

5.4 Zvýšení a snížení hodnoty klíče ve vrcholu

Poslední dvojicí operací, která se nám bude hodit, je zvýšení (INCREASE) a snížení (DECREASE) klíče ve vybraném vrcholu. Předpokládáme přitom, že umíme daný vrchol v haldě přímo najít. Pokud v haldě uchováváme složitější objekty, můžeme si například vést pomocný slovník, který každému objektu přiřadí jeho současnou pozici v haldě. Při každém prohození pak aktualizujeme i slovník. Ten spotřebuje navíc $\mathcal{O}(n)$ paměti.

Realizace samotných operací je již jednoduchá. Pokud provedeme INCREASE klíče v určitém vrcholu, může se halda pokazit pouze směrem dolů. Provedeme tedy BUBBLEDOWN, stejně jako při odebírání kořene.

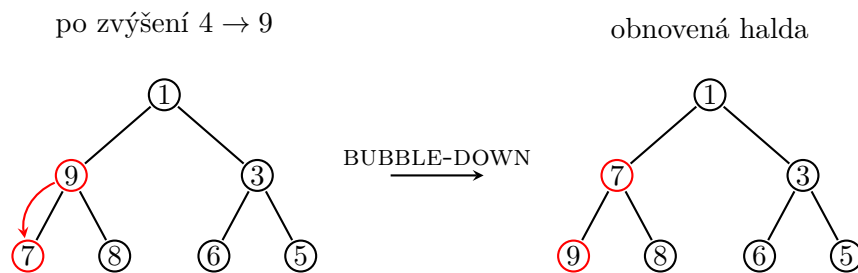


Figure 13: Operace INCREASE: zvýšený klíč se pomocí BUBBLEDOWN přesune dolů.

Operaci DECREASE provedeme zcela stejně, akorát nyní se halda bude kazit ?směrem nahoru?, takže provedeme BUBBLEUP, jako při vkládání nového vrcholu (INSERT).

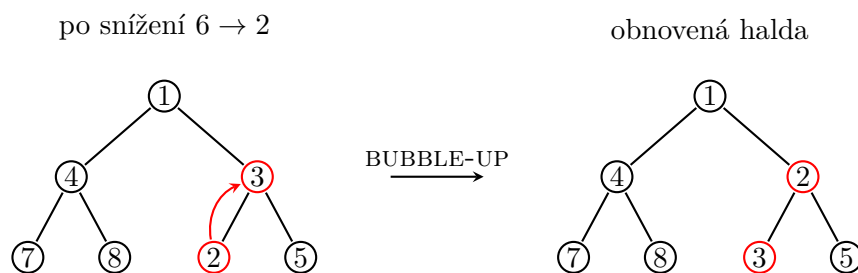


Figure 14: Operace DECREASE: snížený klíč se pomocí BUBBLEUP přesune nahoru.

Pseudokód obou operací zároveň kontroluje, že nový klíč skutečně odpovídá názvu operace. Kdybychom například v operaci INCREASE předali menší hodnotu, bylo by potřeba opravovat haldu opačným směrem.

Algoritmus 5.5: INCREASE**Vstup:** Minimová halda $H[1], \dots, H[n]$, index i a nový klíč x **pokud** $x < H[i]$ **pak**

Ohlas chybu

 $H[i] \leftarrow x$ Zavolej BUBBLE-DOWN(H, i)

Algoritmus 5.6: DECREASE

Vstup: Minimová halda $H[1], \dots, H[n]$, index i a nový klíč x

pokud $x > H[i]$ **pak**

 Ohlas chybu

$H[i] \leftarrow x$ Zavolej BUBBLE-UP(H, i)

V případě *maximové binární haldy* by operace vypadaly zcela stejně, akorát při operaci INCREASE budeme opravovat haldu směrem nahoru a u DECREASE směrem dolů.

5.5 Implementace v Pythonu

Program 5.1 ukládá minimovou haldu v seznamu indexovaném od nuly. Proto používá vztahy $2i + 1$ a $2i + 2$ pro syny a $\lfloor (i-1)/2 \rfloor$ pro rodiče. Metody `bubble_up` a `bubble_down` přesně odpovídají pomocným operacím z pseudokódu; liší se pouze posunutými indexy.

Konstruktor vytváří haldu opakovaným vkládáním, což je didakticky přímočaré, i když pro velké pole existuje rychlejší hromadná konstrukce. Metody `increase` a `decrease` očekávají index prvku. V praktickém programu bychom při ukládání složitějších objektů přidali pomocný slovník objekt–index a aktualizovali jej při každém prohození.

Program 5.1: Minimová binární halda a její základní operace v Pythonu.

```
1  """Minimova binarni halda ulozena v poli indexovanem od nuly."""
2
3
4  class MinHeap:
5      def __init__(self, values=()):
6          self.data = []
7          for value in values:
8              self.insert(value)
9
10     def _parent(self, index):
11         return (index - 1) // 2
12
13     def _left(self, index):
14         return 2 * index + 1
15
16     def bubble_up(self, index):
17         while index > 0:
18             parent = self._parent(index)
19             if self.data[parent] <= self.data[index]:
20                 break
21             self.data[parent], self.data[index] = (
22                 self.data[index], self.data[parent]
23             )
24             index = parent
25
26     def bubble_down(self, index):
27         size = len(self.data)
28         while self._left(index) < size:
29             left = self._left(index)
30             right = left + 1
31             smaller = left
```



```

32         if right < size and self.data[right] < self.data[left]:
33             smaller = right
34         if self.data[index] <= self.data[smaller]:
35             break
36         self.data[index], self.data[smaller] = (
37             self.data[smaller], self.data[index]
38         )
39         index = smaller
40
41     def insert(self, value):
42         self.data.append(value)
43         self.bubble_up(len(self.data) - 1)
44
45     def minimum(self):
46         if not self.data:
47             raise IndexError("Halda je prazdna.")
48         return self.data[0]
49
50     def extract_min(self):
51         if not self.data:
52             raise IndexError("Halda je prazdna.")
53         result = self.data[0]
54         last = self.data.pop()
55         if self.data:
56             self.data[0] = last
57             self.bubble_down(0)
58         return result
59
60     def increase(self, index, new_value):
61         if new_value < self.data[index]:
62             raise ValueError("Nova hodnota musi byt alespon puvodni.")
63         self.data[index] = new_value
64         self.bubble_down(index)
65
66     def decrease(self, index, new_value):
67         if new_value > self.data[index]:
68             raise ValueError("Nova hodnota musi byt nejvyse puvodni.")
69         self.data[index] = new_value
70         self.bubble_up(index)
71
72
73 if __name__ == "__main__":
74     heap = MinHeap([1, 4, 3, 7, 8, 6, 5])
75     print("Halda:", heap.data)
76     heap.increase(1, 9)
77     print("Po increase:", heap.data)
78     heap.decrease(5, 2)
79     print("Po decrease:", heap.data)
80     print("Odebrane minimum:", heap.extract_min())
81     print("Vysledna halda:", heap.data)

```

5.6 Časové složitosti operací

Pojďme si nyní rozebrat časové složitosti zmíněných operací INSERT, MIN, EXTRACTMIN, INCREASE a DECREASE a podívat se, jak moc jsme si pomohli oproti práci s polem (seznamem). Pokud se

pozorně podíváme na operace popsané výše, je zjevné, že nejvíce bude naše operace zpomalovat ono ?probublávání? vrcholu při opravování haldy (vyjma operace MIN, kde jen přečteme klíč v kořeni), tj. BUBBLEUP a BUBBLEDOWN. Na nich bude celková časová složitost záviset.

Všimněme si, že po každém prohození vrcholů při BUBBLEUP nebo BUBBLEDOWN se vrchol posune o jednu hladinu výše/níže.

Víme tak, že při opravování haldy se vrchol může posunout *maximálně o počet hladin*, který má daná halda. Časová složitost daných operací bude tak závislá na jejich počtu. Tím jsme se dostali o trochu blíže řešení, ale rádi bychom věděli, jak se bude dobře trvání daných operací měnit v závislosti na *počtu vrcholů v haldě*, nikoliv počtu hladin. Mezi těmito proměnnými však existuje hezká závislost.

Dejme tomu, že naše halda má n vrcholů v celkově h hladinách. Halda s jednou plnou hladinou má jeden vrchol, se dvěma plnými hladinami tři vrcholy, se třemi sedm vrcholů atd. (viz tabulka 2). Je-li všech

Počet hladin h	1	2	3	4	5	6
Počet vrcholů n	1	3	7	15	31	63

Table 2: Vztah mezi počtem hladin h a počtem vrcholů n v plně obsazené haldě.

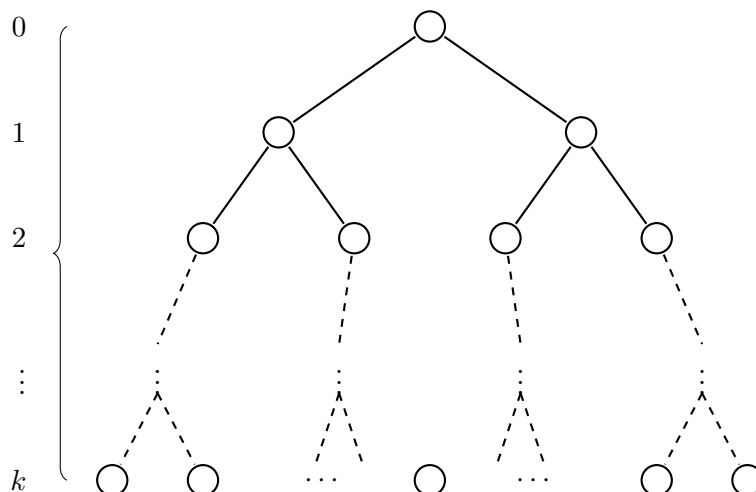


Figure 15: Plná binární halda s obecně h hladinami.

h hladin plných, platí $n = 2^h - 1$. V obecné haldě jsou první $h - 1$ hladiny plné a poslední obsahuje alespoň jeden vrchol. Proto

$$2^{h-1} \leq n \leq 2^h - 1.$$

Z levé nerovnosti dostáváme $h \leq \log_2 n + 1$, tedy počet hladin je $\mathcal{O}(\log n)$. Tím máme vše připravené k odvození složitosti operací.

Věta 5.2 (Složitost operací v binární haldě). Operace INSERT, EXTRACTMIN, INCREASE a DECREASE mají časovou složitost $\mathcal{O}(\log n)$. Operace MIN má časovou složitost $\mathcal{O}(1)$.

Proof. V případě MIN jen přečteme klíč v kořeni, což potrvá $\mathcal{O}(1)$. U zbývajících operací opravujeme haldu pomocí BUBBLEUP nebo BUBBLEDOWN. Při každém prohození se zpracováváný klíč posune o jednu hladinu. Provede se tedy nejvýše $h - 1$ prohození. Protože $h \leq \log_2 n + 1$, dostáváme

$$\mathcal{O}(h) = \mathcal{O}(\log n).$$

Operace INSERT, EXTRACTMIN, INCREASE a DECREASE proto mají logaritmickou časovou složitost. $\square$

Zkusme si na závěr udělat menší porovnání oproti poli, se kterým jsme začínali (viz tabulka 3). Logaritmická časová složitost je určitě lepší, než lineární, je však vidět, že jsme si zároveň pohoršili

	INSERT	MIN	EXTRACTMIN	INCREASE	DECREASE
Pole (seznam)	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Binární halda	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 3: Porovnání časových složitostí operací v haldě a v poli.

v případě vkládání a úpravy hodnot klíčů. Je tomu tak z důvodu, že pole má svou výhodu v téměř nulové režii, kdežto halda (kvůli své pevně definované struktuře) spotřebuje nějaký čas, aby byla uvedena do korektního stavu.

6 Dijkstrův algoritmus

Již jsme si ukázali, že v *neohodnoceném* grafu $G = (V, E)$ umíme relativně jednoduše najít délku nejkratší cesty z výchozího vrcholu $v_0 \in V$ do ostatních vrcholů. Hrany však nemusí mít stejnou cenu. Města mohou například představovat vrcholy a silnice mezi nimi hrany, jejichž váhou bude délka nebo doba průjezdu.

Zde již nestačí počítat počet hran. Na obrázku 16 je cesta (v_0, v_1, v_2) levnější než přímá cesta (v_0, v_2) , přestože obsahuje více hran.

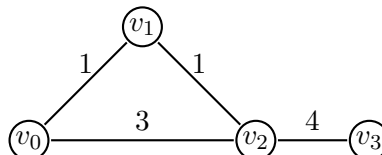


Figure 16: Příklad ohodnoceného grafu.

Pro kladné celočíselné váhy se nabízí převést ohodnocený graf na neohodnocený tak, že hranu váhy k nahradíme cestou o k hranách. Na nově vzniklý graf již lze aplikovat např. BFS, které jsme popsali v sekci 3. Ne každý graf však musí mít „malé“ váhy hran. Pokud bychom vzali např. graf, kde váhy hran jsou v řádech tisíců, bude pro BFS potřeba provést zbytečně mnoho práce, neboť pro zpracování jedné ohodnocené hrany bude třeba vytvořit a projít tisíce hran.

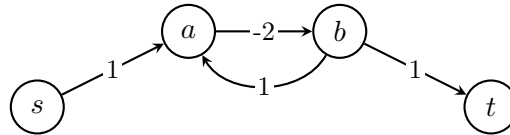
Jedním z nejznámějších algoritmů pro tuto úlohu je *Dijkstrův algoritmus*, který popsal v roce 1959 nizozemský informatik *Edsger W. Dijkstra*.⁶ Podobně jako u BFS a DFS budeme postupně otevírat a uzavírat vrcholy, přičemž každý uzavřeme nejvýše jednou.

První nalezená cesta do libovolného vrcholu již nemusí být nutně nejkratší. Cestu do daného vrcholu bude třeba postupně vylepšovat.

⁽⁶⁾Dijkstrův stručný článek vyšel roku 1959; samotný algoritmus Dijkstra navrhl již roku 1956. Jeho příjmení se přibližně vyslovuje „dajkstra“.

Ohodnocený graf budeme v souladu s prezentacemi zapisovat jako $G = (V, E, w)$, kde $w : E \rightarrow \mathbb{R}$ je váhová funkce. Je-li hrana určena koncovými vrcholy u, v , píšeme pro stručnost $w(u, v)$.

Zaměříme se na grafy, jejichž váhy hran jsou nezáporné. Dijkstrův algoritmus pro graf se zápornou hranou obecně nefunguje, a to ani tehdy, když graf neobsahuje záporný cyklus. Pokud navíc na cestě k cíli leží dosažitelný cyklus se zápornou celkovou vahou, nejkratší cesta konečné délky vůbec neexistuje. Na obrázku 17 lze cyklus (a, b, a) projít libovolněkrát a každým průchodem cenu cesty snížit o 1. Infimum cen cest z s do t je proto $-\infty$. Podobným grafům se tak chceme vyhnout, neboť by naše úvahy pak již



$$d(s, t) = -\infty$$

Figure 17: Graf, kde mezi v_0 a v_4 neexistuje (konečná) nejkratší cesta.

nemusely fungovat (cestu mezi vrcholy bychom mohli potenciálně do nekonečně vylepšovat a algoritmus by se tak nikdy nezastavil).

Algoritmus 6.1: DIJKSTRA

Vstup: Nezáporně ohodnocený graf $G = (V, E, w)$ a počáteční vrchol $v_0 \in V$

// Inicializace

pro každý vrchol $v \in V$ **opakuj**

$stav(v) \leftarrow \text{nenalezený}$

$D(v) \leftarrow \infty$

$stav(v_0) \leftarrow \text{otevřený}$

$D(v_0) \leftarrow 0$

dokud existují otevřené vrcholy **opakuj**

// Kandidát na nejkratší cestu do sousedních vrcholů

$v \leftarrow$ otevřený vrchol s nejmenším D

pro každý soused $s \in V$ vrcholu v **opakuj**

// Našli jsme lepší cestu

pokud $stav(s) \neq \text{uzavřený}$ a $D(v) + w(v, s) < D(s)$ **pak**

$D(s) \leftarrow D(v) + w(v, s)$

$stav(s) \leftarrow \text{otevřený}$

$stav(v) \leftarrow \text{uzavřený}$

Výstup: Seznam vzdáleností D

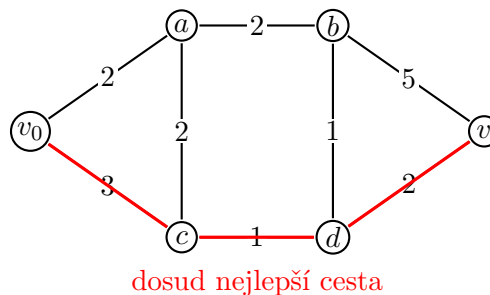


Figure 18: Znázornění situace v průběhu Dijkstrova algoritmu.

6.1 Správnost Dijkstrova algoritmu

Nejprve je dobré si uvědomit, proč algoritmus funguje. Hodnota $D(v)$ je vždy cenou nějaké již nalezené cesty z v_0 do v , a proto nemůže být menší než skutečná nejkratší vzdálenost. Zbývá ukázat opačnou nerovnost ve chvíli, kdy vrchol uzavíráme.

Nejkratší vzdálenost mezi vrcholy u a v označme $d(u, v)$. Důkaz vedeme indukcí podle pořadí uzavíraných vrcholů. Předpokládejme pro spor, že v je první uzavíraný vrchol, pro který $D(v) > d(v_0, v)$. Vezměme jednu nejkratší cestu z v_0 do v . Na této cestě musí existovat první dosud neuzavřený vrchol x ; jeho předchůdce y už uzavřený je. Podle indukčního předpokladu platilo při uzavření y rovnost $D(y) = d(v_0, y)$. Algoritmus tehdy zkontroloval hranu (y, x) , a proto nastavil

$$D(x) \leq d(v_0, y) + w(y, x) = d(v_0, x) \leq d(v_0, v) < D(v).$$

Využili jsme zde nezápornost hran: vzdálenost podél nejkratší cesty se směrem k vrcholu v nemůže zmenšovat. Vrchol x je tedy otevřený a má menší hodnotu D než v . Algoritmus by proto vybral x , nikoli v , což je spor.

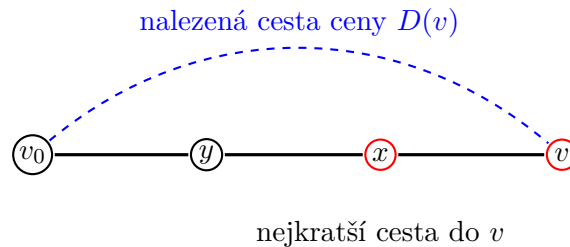


Figure 19: Situace při výběru vrcholu v , kdy $D(v)$ neodpovídá délce nejkratší cesty.

6.2 Časová složitost Dijkstrova algoritmu

Zbývá nám ještě analyzovat časovou složitost. Pokud se zpětně podíváme na pseudokód algoritmu, lze si všimnout, že podstatnou roli bude hrát vybírání vrcholu s minimální hodnotou $D(v)$. Nabízí se tak otázka, jak danou operaci implementovat. První variantou je hodnoty D realizovat jako prostý *seznam* (popř. *pole*).

Věta 6.1 (Dijkstra se seznamem). Dijkstrův algoritmus, kde hodnoty D ukládáme do seznamu (pole), doběhne v čase $\mathcal{O}(n^2 + m)$.

Proof. • Inicializace zabere $\mathcal{O}(n)$ (máme n vrcholů).

- Každý vrchol uzavřeme opět nejvýše jednou, tzn. vnější cyklus se provede $\mathcal{O}(n)$ -krát.
- Hledání minimálního D zabere v rámci každé iterace čas $\mathcal{O}(n)$.
- Stejně jako u BFS a DFS, i zde každou hranu zkontroluji nejvýše dvakrát (záleží, zda graf G je orientovaný), tzn. $\mathcal{O}(2m) = \mathcal{O}(m)$.

Celkově tedy máme

$$\underbrace{\mathcal{O}(n)}_{\text{Init}} + \underbrace{\mathcal{O}(n) \cdot \mathcal{O}(n)}_{\text{Výběr minima}} + \mathcal{O}(m) = \mathcal{O}(n^2 + n + m) = \mathcal{O}(n^2 + m).$$

□

Kvadratická časová složitost není úplně zlá, ale můžeme si ještě trochu přilepšit. Pokud si vzpomeneme na binární haldy z oddílu 5, všimneme si v konečném důsledku, že při použití v Dijkstrově algoritmu se některé operace zrychlí.

Věta 6.2 (Dijkstra s haldou). Dijkstrův algoritmus, kde hodnoty D ukládáme do binární haldy, doběhne v čase $\mathcal{O}((n + m) \cdot \log n)$.

Proof. Inicializace trvá zase $\mathcal{O}(n)$. U operací s binární haldou víme, jak dlouho potrvají (viz tabulka 3). Musíme si jen rozmyslet, kdy se která operace provádí.

- Při prvním *otevření* vrcholu provedeme v haldě operaci INSERT. Celkově vložíme nejvýše všech n vrcholů, což potrvá $\mathcal{O}(n \log n)$.
- Při *uzavírání* vrcholu provádíme operaci EXTRACTMIN (opět max. n -krát), tj. $n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$.
- Při každé další *aktualizaci vzdálenosti* $D(v)$ provádíme DECREASE (tato hodnota se nikdy nemůže zvýšit). Aktualizací je nejvýše tolik, kolik kontrol hran, tedy $\mathcal{O}(m)$. Dostáváme nejvýše $m \cdot \mathcal{O}(\log n) = \mathcal{O}(m \log n)$.

Po sečtení dostaneme

$$\begin{aligned} \mathcal{O}(n + n \log n + n \log n + m \log n) &= \mathcal{O}(n + 2n \log n + m \log n) \\ &= \mathcal{O}(n + n \log n + m \log n) \\ &= \mathcal{O}(n \log n + m \log n) \\ &= \mathcal{O}((n + m) \cdot \log n). \end{aligned}$$

□

Ačkoliv to není úplně zjevné, výraz $(n + m) \cdot \log n$ roste o dost pomaleji oproti $n^2 + m$, tedy binární haldou si skutečně pomůžeme oproti seznamu.

Závěrem uvedme, že často nehledáme cestu do všech vrcholů, ale pouze mezi konkrétní dvojicí. Výpočet můžeme zastavit ve chvíli, kdy cílový vrchol vybereme jako minimum a uzavřeme jej: tehdy už je jeho hodnota D konečná. Hledání lze v některých situacích dále urychlit, protože Dijkstrův algoritmus může prozkoumávat i vrcholy směřující úplně jinam než k cíli. Na to se podíváme v další sekci 7.

7 Algoritmus A*

Nyní se omezíme na případy, kdy hledáme cestu pouze do konkrétního cílového vrcholu $g \in V$ ⁷. Stále pracujeme s grafy, jejichž hrany mají nezáporné ohodnocení. Dijkstrův algoritmus by samozřejmě fungoval i zde: skončili bychom ve chvíli, kdy cílový vrchol vybereme jako minimum. Nabízí se však otázka, zda nemůžeme znalost cíle nějak využít. Pokud bychom například hledali nejkratší cestu v mřížce s překážkami, dávalo by smysl upřednostňovat políčka, která vypadají být cíli blíž.

Mějme bludiště, v němž se hráč může pohybovat o jedno pole vodorovně nebo svisle; na některá pole vstoupit nesmí. Cíl se nachází vpravo dole. Situaci na obrázku 20 můžeme znázornit grafem, v němž každá přípustná dvojice sousedních polí tvoří hranu s vahou 1. Pokud bychom použili BFS nebo Dijkstrův algoritmus, prozkoumávali bychom okolí startu rovnoměrně do všech směrů. Algoritmus A*, který roku 1968 popsali Peter Hart, Nils Nilsson a Bertram Raphael, se snaží hledání směřovat k cíli.

⁽⁷⁾ Písmeno g od anglického *goal*.

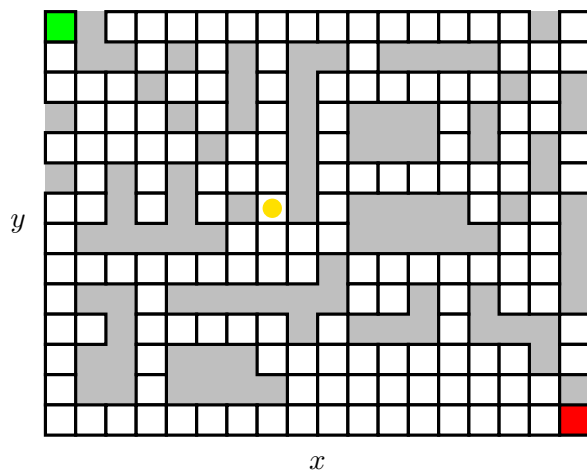


Figure 20: Mřížka a optimální směr pohybu hráče.

Myšlenka je stále podobná, avšak vrcholy budeme vybírat podle hodnoty $D(v) + \psi(v)$ místo samotného $D(v)$. Zobrazení $\psi : V \rightarrow \mathbb{R}_0^+$ se nazývá *heuristická funkce* nebo zkráceně *heuristika* a odhaduje zbývající vzdálenost z vrcholu v do cíle (viz obrázek 21). Součtu $D(v) + \psi(v)$ budeme říkat *f-skóre*⁸, značíme jej

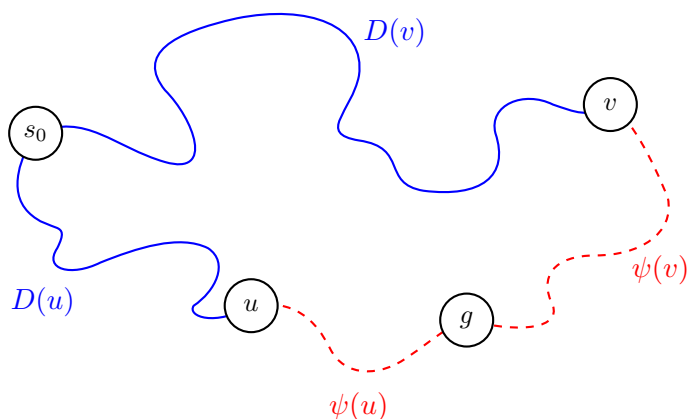


Figure 21: Znázornění heuristické funkce ψ .

$f(v)$. V každé iteraci vybereme otevřený vrchol s nejnižším f -skóre.

Ne každá heuristika zachová správnost této strategie. Budeme požadovat

$$\psi(g) = 0 \quad \text{a} \quad \psi(u) \leq w(u, v) + \psi(v)$$

pro každou orientovanou hranu $(u, v) \in E$. Takové heuristice říkáme *konzistentní*. Druhá podmínka je obdobou trojúhelníkové nerovnosti: odhad z u nesmí být větší než cena jednoho kroku do v a odhad z v . Opakovaným použitím podél libovolné cesty z u do cíle dostaneme, že $\psi(u)$ nikdy nepřeceňuje skutečnou nejkratší vzdálenost do cíle.

Oproti Dijkstrovu algoritmu tedy nenastává mnoho změn, jen si navíc uchováváme hodnoty f -skóre.

⁽⁸⁾V anglických textech se pro dosud nalezenou vzdálenost často používá označení g a pro heuristiku h , tedy $f(v) = g(v) + h(v)$.

Algoritmus 7.1: A*

Vstup: Nezáporně ohodnocený graf $G = (V, E, w)$, vrcholy $v_0, g \in V$ a konzistentní heuristika ψ
// Inicializace

pro každý vrchol $v \in V$ **opakuj**

$stav(v) \leftarrow \text{nenalezený}$

$D(v) \leftarrow \infty, f(v) \leftarrow \infty$

$stav(v_0) \leftarrow \text{otevřený}$

$D(v_0) \leftarrow 0, f(v_0) \leftarrow \psi(v_0)$

dokud existují otevřené vrcholy **opakuj**

$v \leftarrow$ otevřený vrchol s nejmenším f -skóre

pokud $v = g$ **pak**

 vrať $D(g)$

pro každý soused s vrcholu v **opakuj**

 // Našli jsme lepší cestu

pokud $stav(s) \neq \text{uzavřený}$ a $D(v) + w(v, s) < D(s)$ **pak**

$D(s) \leftarrow D(v) + w(v, s)$

$f(s) \leftarrow D(v) + w(v, s) + \psi(s)$

$stav(s) \leftarrow \text{otevřený}$

$stav(v) \leftarrow \text{uzavřený}$

vrať ∞

Výstup: Vzdálenost v_0 od g

7.1 Správnost algoritmu A*

Podobně jako u Dijkstrova algoritmu platí, že při výběru vrcholu v odpovídá $D(v)$ délce nejkratší cesty z v_0 do v . Důkaz provedeme indukcí podle pořadí uzavírání vrcholů.

Předpokládejme pro spor, že vybíráme první vrchol v , pro který je $D(v) > d(v_0, v)$. Na jedné nejkratší cestě do v vezmeme první dosud neuzavřený vrchol x a jeho již uzavřeného předchůdce y . Podle indukčního předpokladu bylo $D(y) = d(v_0, y)$, takže po zpracování hrany (y, x) platí $D(x) = d(v_0, x)$. Konzistenci můžeme opakovaně použít na část této cesty od x do v . Dostaneme

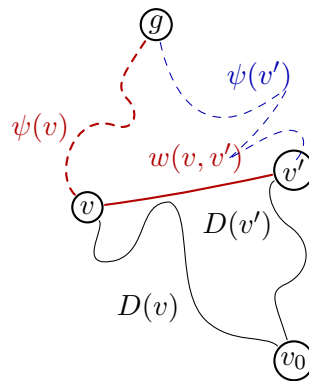


Figure 22: Situace při výběru vrcholu v při heuristice ψ , kdy $D(v)$ neodpovídá délce nejkratší cesty.

$$\psi(x) \leq d(v_0, v) - d(v_0, x) + \psi(v).$$

Proto

$$f(x) = D(x) + \psi(x) \leq d(v_0, x) + d(v_0, v) - d(v_0, x) + \psi(v) = d(v_0, v) + \psi(v) < D(v) + \psi(v) = f(v).$$

Vrchol x je otevřený a má menší f -skóre než v , takže algoritmus měl vybrat x . To je spor. Každý uzavřený vrchol tedy dostane správnou vzdálenost; zvlášť to platí pro cílový vrchol g .

Konzistence heuristiky je pro uvedenou variantu algoritmu důležitá. Pokud by odhad tuto podmínku porušoval a algoritmus uzavřené vrcholy znovu neotevíral, mohl by některý vrchol uzavřít předčasně a vrátit neoptimální cestu.

Poznámka 7.1. • Dijkstrův algoritmus je speciálním případem A^* . Stačí pro všechny vrcholy v položit $\psi(v) = 0$.

- Heuristiku bychom měli být schopni pro každý vrchol spočítat rychle. Ideální je konstantní čas $\mathcal{O}(1)$.

V nejhorším případě může A^* prozkoumat celý graf, takže při použití binární haldy má stejný asymptotický horní odhad jako Dijkstrův algoritmus, tedy $\mathcal{O}((n + m) \log n)$. Dobrá heuristika však obvykle výrazně sníží počet skutečně zpracovaných vrcholů. Volba $\psi = 0$ neposkytne žádné směřování a dostaneme přesně Dijkstrův algoritmus.

7.2 Ukázky heuristických funkcí

Zbývá zodpovědět otázku, jakou heuristiku použít. Odpověď záleží na konkrétní úloze. Vraťme se k bludišti, v němž se hráč pohybuje vodorovně nebo svisle o jedno pole. Políčka identifikujeme souřadnicemi (x, y) .

Jednou možností je tzv. *eukleidovská vzdálenost*. Ta je pro libovolnou dvojici bodů v rovině $X = (x_1, y_1)$ a $Y = (x_2, y_2)$ definována jako

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Známe-li souřadnice cílového políčka, můžeme eukleidovskou vzdálenost použít jako heuristiku ψ_E . Překážky ignorujeme, takže dostaneme dolní odhad skutečné délky cesty. Situaci lze sledovat na obrázku 23. Protože se v našem bludišti nelze pohybovat po diagonálách, ještě přirozenější volbou je *manhattanská*

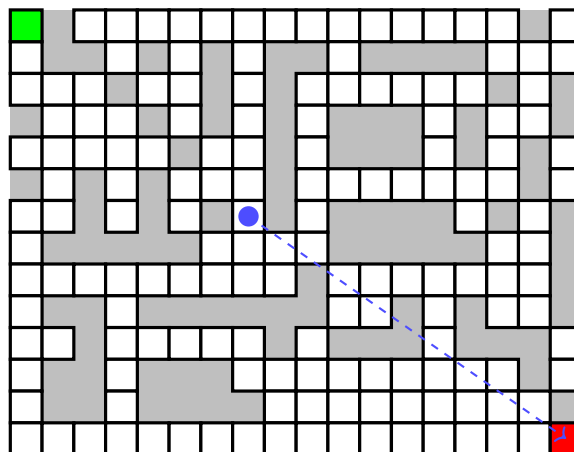


Figure 23: Příklad bludiště s eukleidovskou heuristikou.

vzdálenost ψ_M definovaná vztahem

$$|x_1 - x_2| + |y_1 - y_2|.$$

Eukleidovská i manhattanská vzdálenost splňují trojúhelníkovou nerovnost, a proto v této mřížce

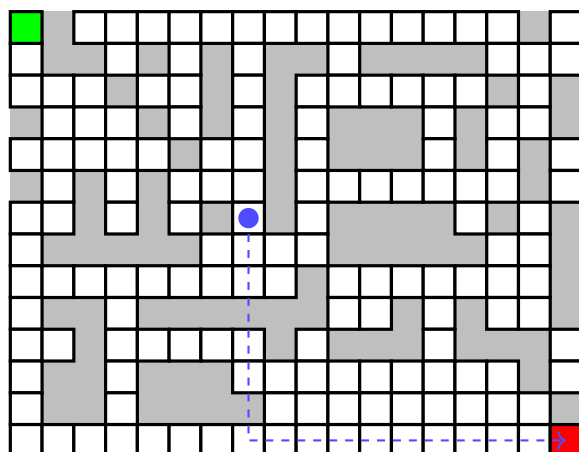


Figure 24: Příklad bludiště s manhattanskou heuristikou.

dávají konzistentní heuristiky. Manhattanská vzdálenost je zde navíc přesnější: bez překážek se rovná skutečnému počtu vodorovných a svislých kroků do cíle. Její větší hodnota tedy není nevýhodou, dokud nepřeceňuje skutečnou cenu cesty. Naopak zpravidla lépe směřuje hledání.